WEB TECHNOLOGIES FINAL PROJECT

Four Seasons Across China Final Project Report

An interactive website built with pure Python, HTML, CSS, and JavaScript

This report explains the design, implementation, testing, and final result of a four-season interactive website created for the Web Technologies course.

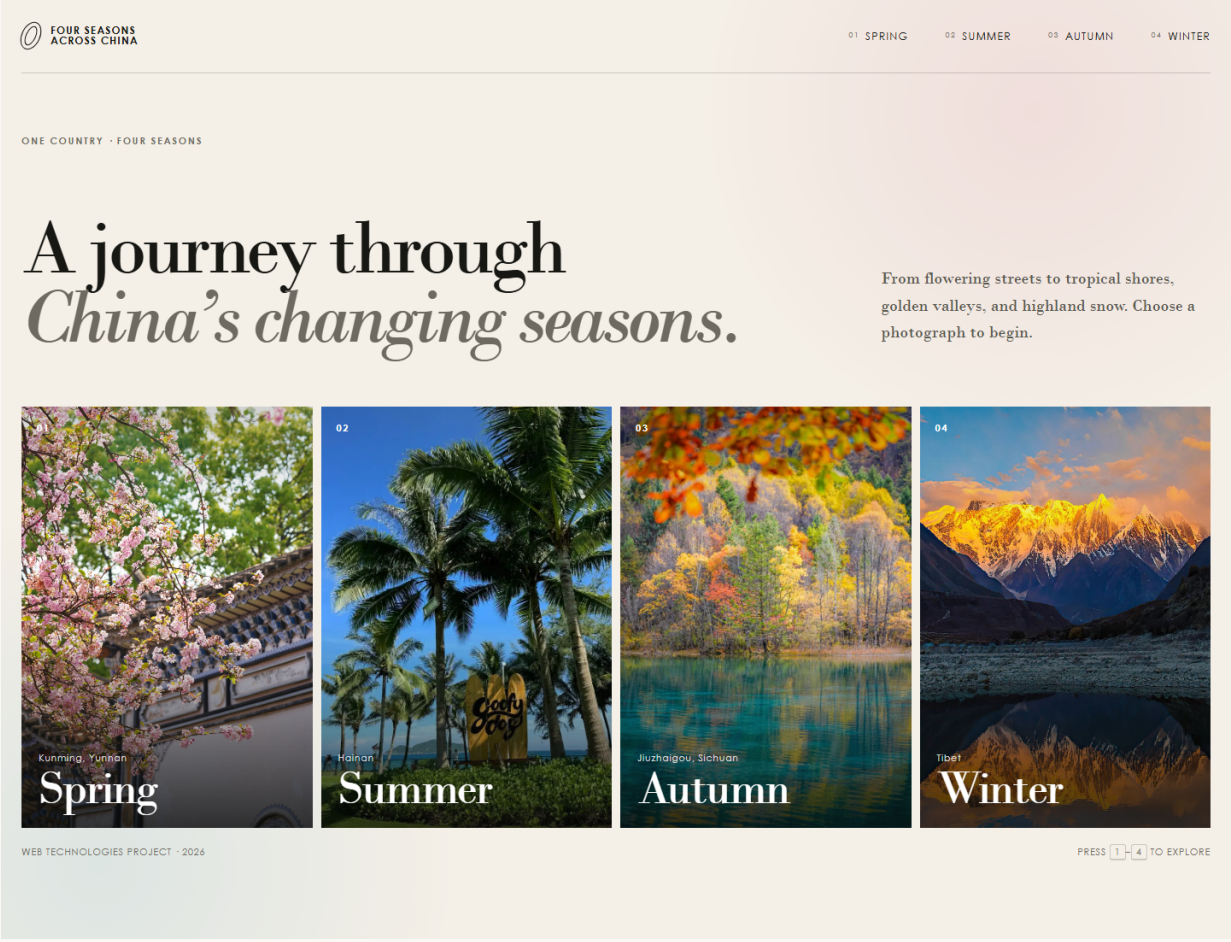


Figure 1. The four-season homepage in the final desktop layout.

Project Overview

Four Seasons Across China is an interactive website created for my Web Technologies final project. The site presents one representative landscape for each season: spring in Kunming, summer in Hainan, autumn in Jiuzhaigou, and winter in Tibet. The homepage gives each season a photographic entry point. Clicking an image opens a larger photograph, an English quotation, and the author's name.

The final version uses Python, HTML, CSS, and JavaScript. Python is implemented with the standard library only. The website does not require Flask, Jinja, a database, or any third-party package. This choice keeps the project easy to run and makes every main part of the request flow visible in a small number of files.

Project Goals

The project has four practical goals:

- Make the main interaction understandable without a long instruction.
- Give each season a distinct visual identity through place, photograph, and colour.
- Use one reusable page structure for all four seasonal pages.
- Practise Python routing, HTML structure, CSS layout, JavaScript interaction, and testing.

Project Content and Website Design

The homepage uses a warm paper background, dark typography, and a four-card image grid. Each card has a strong vertical format so that the photograph remains the main visual element. On hover or keyboard focus, the image enlarges slightly and a small arrow appears.

Season	Location	Visual focus	Author
Spring	Kunming, Yunnan	Cherry blossoms and traditional architecture	Ralph Waldo Emerson
Summer	Hainan	Palm trees, blue sky, and the coast	Sam Keen
Autumn	Jiuzhaigou, Sichuan	Autumn forest reflected in water	Albert Camus
Winter	Tibet	Snow mountains and reflected light	Percy Bysshe Shelley

Each seasonal page has two main areas on desktop screens. The photograph occupies the larger left area, and the quotation occupies a colour field on the right. Previous and next links at the bottom allow continuous exploration. On a phone, the layout becomes a single vertical column with the image first.

User Journey

- The visitor lands on a single page with four clearly labelled season cards.
- The visitor selects a photograph and enters the corresponding season page.
- The visitor reads the quotation, moves to another season, or returns to the overview.
- Keyboard and touch alternatives support movement without requiring a mouse.

The photographs act as the main navigation. This follows the original project idea: a simple website in which a visitor clicks an image and enters that season.

System Architecture and Python Implementation

The project has three layers. The browser displays the interface, the Python application prepares each response, and local files store the seasonal content and visual assets.

Layer	Responsibility	Implementation
Client	Displays the interface and handles short interactions	HTML, CSS, JavaScript
Python application	Chooses pages, prepares HTML, and serves files	http.server and pathlib
Content	Stores seasonal text and image paths	Python dictionaries and local files

When the browser requests the homepage, Python returns the index template. When it requests /season/spring, Python reads the shared season template, inserts the spring values, and returns the completed page. Requests beginning with /static/ return CSS, JavaScript, or image files with the correct content type.

Template Rendering

All seasonal content is stored in the SEASONS dictionary inside app.py. The dictionary contains the page name, location, image, alternate text, quotation, author, and colours. A small function replaces named placeholders in the shared HTML template.

```
def render_template(template_name, values):
    template = (TEMPLATE_DIR / template_name).read_text(encoding="utf-8")
    for key, value in values.items():
        template = template.replace("{{" + key + "}}", str(value))
    return template
```

The SiteHandler class extends BaseHTTPRequestHandler. It supports GET and HEAD requests, sets the response type, checks static paths, and sends a custom 404 response when a route does not exist. The server reads the PORT environment variable, so the same application can run locally or on a hosting platform.

Interaction Responsive Design and Accessibility

The site uses JavaScript for small enhancements. Number keys one through four open the corresponding season from the homepage. Arrow keys move between seasons, and Escape returns to the homepage. A horizontal swipe changes seasons on touch screens. Normal links remain available, so the core navigation still works if JavaScript is unavailable.

- Every photograph has descriptive alternative text.
- Cards and links have visible keyboard focus outlines.
- A reduced-motion media query removes most transitions when requested.
- Responsive CSS rearranges the page without separate mobile HTML files.



Figure 2. The spring page at a mobile viewport width.

The mobile layout places the photograph first and the quotation below it. Important parts of the image remain visible through object positioning, while the text width and type size change to avoid horizontal scrolling.

Testing and Results

I tested the Python responses and the visual layout. The six automated tests use Python's built-in unittest module and start the actual HTTP server on a temporary local port.

Test	What it checks	Result
Homepage test	All four seasons appear	Passed
Season page tests	All four pages return HTTP 200	Passed
Invalid route test	Unknown season returns HTTP 404	Passed
API test	JSON endpoint returns four records	Passed
Static image test	Image returns HTTP 200 and JPEG type	Passed
Health check	Server reports that it is running	Passed

The command python -m unittest discover -s tests -v runs all six tests. I also checked the homepage and all four season pages in a desktop browser and reviewed the season layout at a mobile viewport. The screenshots in this report show the final rendered states.

Design Challenges and Solutions

The first challenge was creating a clear seasonal contrast. A single-location set did not show enough visual change between seasons. Four locations solved this problem while keeping a simple theme: one country and four different seasonal landscapes.

The second challenge was building the site without a framework. The standard-library version keeps the request flow explicit: inspect the URL, prepare content, and send a status, content type, and response body. The third challenge was responding to different image dimensions. CSS object-fit and a small set of object positions keep each subject visible.

Conclusion

The final website meets the main project goal: a visitor can understand the four-season structure quickly, click a photograph, and move through the content without unnecessary steps. Python, HTML, CSS, and JavaScript each have a clear role, and every part is small enough to explain during a presentation.

The project is ready to run locally with one Python command and to be uploaded to GitHub. The included deployment configuration allows the same application to be published as a public web service. The final submission should also include the public repository and demonstration links required by the course.

Final Deliverables

- Complete Python website source code
- English project report
- Desktop and mobile preview images
- Six automated tests
- GitHub and deployment instructions

All core features were working in the final local test. The remaining submission step is to upload the source to GitHub, deploy the service, and place the two public links in the course submission form.